

MasteryLS: A Git-Native, AI-Augmented Learning Management System for Mastery-Based Learning Across Disciplines

Lee S. Jensen

Department of Computer Science, Brigham Young University, Provo, UT, USA · lee@cs.byu.edu

ABSTRACT

Learning management systems typically treat artificial intelligence as an add-on to a database-centric content model, and, when they support code-related assessment at all, typically assume a computer-science or STEM audience. We present MasteryLS, a learning management system built on two commitments: course content is stored and versioned in Git repositories rather than a proprietary database, and AI is integrated as a first-class capability across content authoring, feedback, and assessment rather than bolted on afterward. Instructors author Markdown content that AI can draft, expand, and review, while every interaction, from auto-graded multiple-choice questions to open-ended essays and live GitHub-repository grading, is a declarative, inspectable block embedded directly in that content. The system's most novel contribution is Disciplinary Reasoning Assessment (DRA), a generative simulation framework in which an instructor authors only high-level parameters (discipline, problem type, difficulty, learning outcomes) and the AI generates, at each learner's runtime, a full professional scenario with stakeholders, resources, and constraints that the learner investigates through six universal disciplinary stages (Frame, Research, Model, Act, Validate, Reflect); evaluation is grounded in evidence extracted from the learner's own transcripts rather than a holistic AI judgment. A related Interview topic type reuses the same engine for career-readiness practice. We describe MasteryLS's

architecture and report on its deployment in three computer science courses at Brigham Young University, alongside prototype exploration of the framework in six additional disciplines. This is a systems and experience paper: we report design and deployment, not a controlled outcomes study, and we discuss what evaluation work is needed next.

1 INTRODUCTION

Course content authoring and formative feedback remain two of the most persistent bottlenecks in higher education, and neither problem is specific to computer science. Instructors across disciplines spend disproportionate time producing and revising course material, and learners in any field benefit from prompt, substantive feedback that current staffing models struggle to provide at scale. Meanwhile, mainstream learning management systems (LMS) such as Canvas and Moodle store content in proprietary databases behind web-based editors, offering little of the version control, diffing, and multi-author collaboration that developers take for granted in Git-based workflows, and treat artificial intelligence, where it appears at all, as an external plugin rather than part of the platform's design.

We present MasteryLS, an LMS built around two commitments that run through every layer of the system. First, course content is Markdown stored in GitHub repositories, not a proprietary database: instructors get commit history, diffing, branching, and multi-author collaboration for free, and can move, audit, or fork their content independently of the platform. Second, AI is integrated as a first-class capability at every stage of the content and assessment lifecycle, namely authoring, feedback, and simulated professional practice, rather than as a bolt-on chatbot.

Although this paper is written for a SIGCSE audience and centers its deployment discussion on computer science courses, MasteryLS's content model and assessment framework are explicitly domain-agnostic. Our most novel contribution, Disciplinary Reasoning Assessment (DRA), was designed from the outset as a universal framework usable across disciplines and problem domains, and we report both real classroom deployment in computer science and prototype exploration of the same framework in six other disciplines.

This paper makes the following contributions:

- A Git-native, domain-agnostic content model that brings software-engineering-grade version control to LMS authoring.
- An AI-assisted content-generation pipeline, integrated into the authoring workflow, that produces draft course structure and lesson content from instructor descriptions.
- A declarative interaction model in which AI grading and feedback, including rubric-graded essays, file submissions, and live GitHub-repository grading, is a first-class part of course content rather than a separate grading system.
- Disciplinary Reasoning Assessment and its Interview specialization: a generative, cross-disciplinary scenario-based assessment framework that evaluates process, competency, and disposition from evidence extracted directly from learner transcripts.
- A deployment report covering three real computer science courses and prototype cross-disciplinary exploration in six additional fields.

We are explicit about scope: this is a systems and experience paper. We have not yet collected learner-outcome or satisfaction data, and we discuss this limitation directly in Sections 8 and 9 rather than overstating what our deployment experience shows.

2 RELATED WORK

Traditional LMS platforms such as Canvas and Moodle provide mature gradebook, enrollment, and content-delivery infrastructure, but store content in platform-specific databases and offer only coarse revision history compared to a Git workflow. Git-backed, “docs-as-code” tooling has been explored specifically within computer science education: empirical work describes using Git itself as a course delivery platform [4] and documents student experiences using GitHub in software engineering courses [5]. MasteryLS extends this pattern from a supplementary tool into the LMS's canonical content store, and generalizes it beyond CS-specific repository-based assignments.

A large and fast-growing body of work studies large language models (LLMs) in computer science education specifically, including automated feedback generation, virtual teaching assistants, and code review [3]. MasteryLS's github-submission

interaction (Section 5.2) sits within this literature, but is designed as one interaction type among many in a discipline-agnostic authoring and grading model rather than a CS-specific autograder.

Scenario-based and simulation-based assessment has deep precedent outside CS: standardized-patient simulation has been used to appraise clinical performance in medical education since the 1960s [2], and case-based learning is a staple of business and legal pedagogy. Disciplinary Reasoning Assessment (Section 5.3) draws on this tradition but generalizes it: rather than authoring a fixed scenario per discipline, the framework generates scenarios at runtime from author-specified parameters, and evaluates six universal stages of professional reasoning that are interpreted differently by discipline.

Mastery learning [1] is the pedagogical foundation for MasteryLS's overall design: content mastery, tracked per topic, drives progression rather than a fixed calendar. The DRA and Interview evaluation model builds on evidence that learners who explain material back to a teachable agent expend more effort and learn more, an effect sometimes called the protégé effect [6].

3 SYSTEM ARCHITECTURE

MasteryLS is a client-side single-page application (React, served via Vite) backed by three external systems, each responsible for a distinct concern. GitHub is the canonical store for course content: course structure, topic Markdown, and embedded interaction definitions are stored as files in an ordinary Git repository, with a top-level `course.json` (or, if absent, a discovered `instruction/modules.md`) declaring the module and topic hierarchy. Supabase (PostgreSQL, Auth, and Edge Functions) stores operational data, including users, roles, enrollments, and a fine-grained progress and event log, and hosts server-side Edge Functions that proxy calls to the Gemini API and to the Canvas API, so that AI and third-party credentials never reach the client. An optional integration synchronizes selected topics, quizzes, and grades with an Instructure Canvas course, automatically stripping AI-only functionality (such as DRA and Interview) from the Canvas-facing view, since those interactions have no Canvas equivalent.

This split is deliberate: GitHub content is portable, auditable, and usable independently of MasteryLS, while Supabase holds only the operational state that has no meaningful existence outside a specific deployment. The same split underlies our cross-disciplinary generality claim: nothing in the architecture assumes CS-specific content, code execution, or grading.

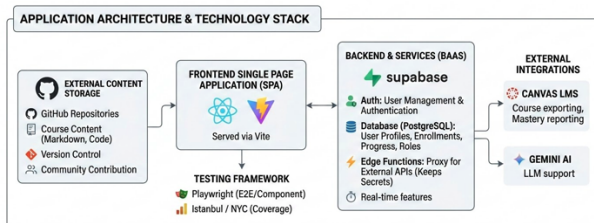


Figure 1. MasteryLS system architecture: a React single page application backed by GitHub (canonical content), Supabase (operational data and Edge Functions), and the Gemini and Canvas APIs.

4 ACCELERATING CONTENT DEVELOPMENT WITH GITHUB AND AI

Because content is Markdown in a Git repository, instructors get commit-level history, diffing, revert, and multi-author

collaboration without any bespoke versioning system. The in-browser editor (Monaco) supports word wrap, line numbers, changed-line markers against the last commit, multi-cursor editing, and search/replace, alongside toolbar shortcuts for common Markdown constructs and for inserting interaction templates.

AI assistance is layered onto this workflow rather than replacing it. At course-creation time, an instructor can supply a title and description and receive an AI-generated starter structure of modules and topic stubs; at the topic level, stub topics can be expanded into first-draft lesson Markdown in batches. Finer-grained tools operate within a topic: generating a new section, rewriting only a selected range of text, running a spelling/grammar/content review pass, and generating an illustrative image. In every case the recommended and observed workflow is draft-review-refine: AI output is treated as a blueprint that an instructor curates, not a final artifact, and the underlying Git history means any AI-authored change can be diffed, reverted, or attributed like any other commit.

We do not yet have timing data comparing AI-assisted to fully manual authoring (Section 8 discusses this as a needed follow-up study). What we do have is breadth: beyond the three deployed computer science courses, the same authoring pipeline has been used, as a prototype exploration, to draft course content for Human-Computer Interaction, Electrical Engineering, Chemical Engineering, Legal, Religious Education, and Agricultural topics. None of these six was deployed to a live course with students, but the pipeline's ability to produce plausible starting material across such a heterogeneous set of domains, from a chemical-engineering process description to a religious-education lesson, is itself evidence that the authoring tools are not tuned to STEM or CS-specific content.

5 AI-DRIVEN FEEDBACK AND ASSESSMENT

5.1 Interaction Model Overview

Every gradable or interactive element in a MasteryLS topic, such as a multiple-choice question, an essay prompt, a file upload, or a live chat, is represented as a single declarative block, fenced with a `masteryls` code fence, embedded directly in the topic's Markdown next to the surrounding instructional text. A minimal JSON header (id, title, type, and type-specific parameters) plus a Markdown body fully defines the interaction; there is no separate authoring surface or database record to keep in sync with the content. This lets us classify interactions along one dimension that recurs throughout the system: auto-scored objective types (multiple-choice, multiple-select) that require no AI at grading time; AI-graded, rubric-driven types (essay, file-submission, url-submission, github-submission, ai-web-page) that accept an optional grading-criteria string used by the model at grading time; and AI-conversational types (teaching, prompt) in which the interaction itself is a dialogue. Two boolean flags, `autoGrade` and `syncGrade`, control whether a score is posted automatically or held for manual submission, and a mentor can always overwrite an AI-generated score or comment. Automation never removes instructor authority over the final grade.

5.2 Deep Feedback Across Interaction Types

Several interaction types are designed to produce feedback that goes well beyond a right or wrong marker. For these types the AI reads free text, a linked resource, an uploaded file, or an entire codebase, and returns qualitative, source-grounded commentary

alongside a score. In every case an optional `gradingCriteria` string lets the instructor specify a rubric that is not necessarily shown to the learner, and the `autoGrade` and `syncGrade` flags described above still apply.

Essay. An essay interaction collects a free response and grades it against the essay prompt, the surrounding topic content, and, when supplied, the additional grading criteria. The returned feedback is written prose that explains what the response did well and what it missed, so a learner receives more than a percentage.

Teaching. A teaching interaction inverts the usual question and answer pattern. Rather than answering a question, the learner explains a concept to the AI across an open dialogue, and the AI evaluates the full conversation, not a single message, for correctness, depth, and how the explanation holds up under follow up questions. Feedback addresses the reasoning shown across the dialogue rather than a single submitted answer.

Url submission. A url-submission interaction collects a link, optionally validates that it is reachable, and, when grading criteria are supplied, has the AI evaluate the content the link points to. An optional `urlPrompt` can transform the submitted URL before evaluation. For example, an instructor can have `MasteryLS` rewrite a repository URL into the raw content path for its README file, so the AI grades the actual rendered text rather than the GitHub page that surrounds it.

File submission. A file-submission interaction accepts one or more uploaded files and applies the same rubric-grounded grading used for essays. A learner can submit a report, a dataset, or a design document and receive feedback on the uploaded artifact itself rather than on a description of it.

GitHub submission. The deepest example is `github-submission`. Where `url-submission` grades a single linked page, `github-submission` has the AI fetch and read the files of an entire public repository and grade them against the instructor's expected outcomes. Application code is weighted over Markdown, and large or binary files are skipped, so the model spends its attention on the parts of the repository that matter to the rubric. The result is a score from 0 to 100 and Markdown feedback that cites real file paths, telling the learner, for a rubric item such as “automated tests cover core behavior,” exactly which files were checked and where they fell short. If an instructor omits grading criteria, the submission scores 0 with an explicit “no grading criteria configured” message, so grading is never silently vague. Because this mechanism evaluates the contents of a codebase rather than a single answer, it generalizes past programming assignments to any discipline whose deliverables are file based, such as a data-analysis notebook, a document set, or a design-file archive.

5.3 Disciplinary Reasoning Assessment (DRA)

DRA is `MasteryLS`'s most novel contribution and its clearest evidence against a CS-specific reading of the system. Rather than authoring a fixed case study, an instructor authoring a DRA topic specifies only generation parameters: a target discipline, a problem type, a difficulty level on a scale from 1 to 5, which of two modes are enabled (Practice, Final, or both), whether random “instability” events are enabled, and the intended learning outcomes. No scenario, stakeholder, or resource is written or reviewed at authoring time. When a learner begins the assessment, the AI generates, from those parameters, a complete scenario: a narrative situation, a set of stakeholders and resources with distinct personalities and bounded knowledge, constraints,

and, depending on difficulty, inflection points and standards of evidence.

The learner works the scenario through six universal disciplinary stages: Frame, Research, Model, Act, Validate, and Reflect, each interpreted differently depending on discipline (Table 1). At every point the learner can select a stakeholder or resource, converse with an AI agent playing that role, and record structured reasoning (current understanding, assumptions, unknowns, hypotheses, decisions, evidence, confidence). Additional stakeholders and resources surface automatically as they are mentioned in conversation, rather than being listed up front, so investigation itself is part of the task. In Practice mode the learner can regenerate or abandon a scenario freely and receives continuous AI coaching; in Final mode the scenario is locked after a single generation and feedback is withheld until completion.

Difficulty is not a single knob but six independently calibrated levers: how much of the scenario is disclosed up front, how specific the per-stage hints are, how cooperative stakeholders are in conversation, how directive practice-mode coaching is, how high the evidence bar is for a strong rating, and whether the generated constraints are mutually satisfiable. All six are set together so the experience shifts coherently from a scaffolded tutorial to an unguided professional challenge.

Evaluation is deliberately evidence-grounded rather than a single holistic AI judgment. An observation agent scores three dimensions: Process (the six stages), Competency (systems thinking, communication, design reasoning, evidence-based reasoning, decision-making), and Disposition (curiosity, ownership, integrity, persistence, empathy, accountability). Every attribute rating is backed by discrete evidence items extracted from the learner's transcripts, each carrying a polarity (positive or negative) and an impact weight, so a high rating with sparse or contradicted evidence cannot sustain a high score. Completed assessments accumulate into an exportable portfolio that shows a trajectory across scenarios rather than a single grade.

Stage	SE	Biology	History	Acct.
Frame	Clarify stakeholders/constraints	Define the biological question	Define the historical problem	Clarify the financial issue
Research	Gather requirements	Collect observations	Examine primary sources	Collect financial records
Model	Design architecture	Develop hypotheses/models	Construct interpretations	Build financial models
Act	Implement software	Conduct experiments	Write historical argument	Prepare statements
Validate	Test behavior	Analyze results	Evaluate evidence	Audit and reconcile
Reflect	Evaluate tradeoffs	Consider limitations	Reconsider interpretations	Assess risks

Table 1. Illustrative interpretation of DRA's six universal stages across four example disciplines, as sketched in the framework's design documentation. This table illustrates intended generality; it is not drawn from a deployed non-CS course (see Section 8).

DRA is not only a design proposal: Figure 2 shows an actual final assessment run in CS 240 (Advanced Software Construction). The generated scenario asks the learner to extend a school grading system, and the workspace view (Figure 2a) shows the learner in the Model stage, consulting an AI-played IT manager stakeholder about API design while recording architecture notes alongside a product owner and a security officer as additional stakeholders and a legacy data export and a

specification draft as resources. Figure 2b shows the resulting evaluation for that same run: an overall score of 92 out of 100, rated Exemplary, computed from a Process score of 100, a Competency score of 96, and a Disposition score of 70, with 42 supporting evidence items such as “identified the 128MB RAM limit for the CLI and estimated the JVM footprint to ensure compliance,” each contributing a small, explicit support weight to its attribute score rather than a single opaque AI judgment. This is one run, not a systematic sample, but it is real deployed evidence rather than a hypothetical illustration.

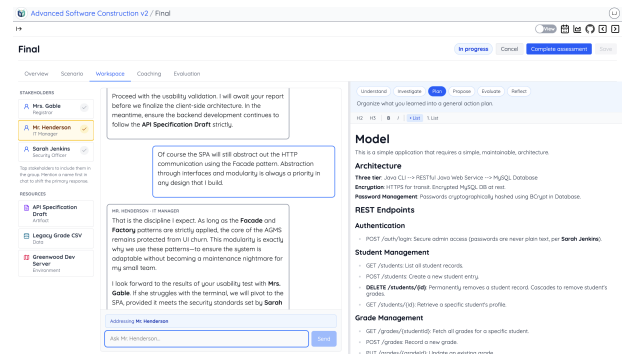


Figure 2a. DRA Workspace view from a CS 240 final assessment: the learner interviews an AI-played IT manager stakeholder while recording notes in the Model stage.

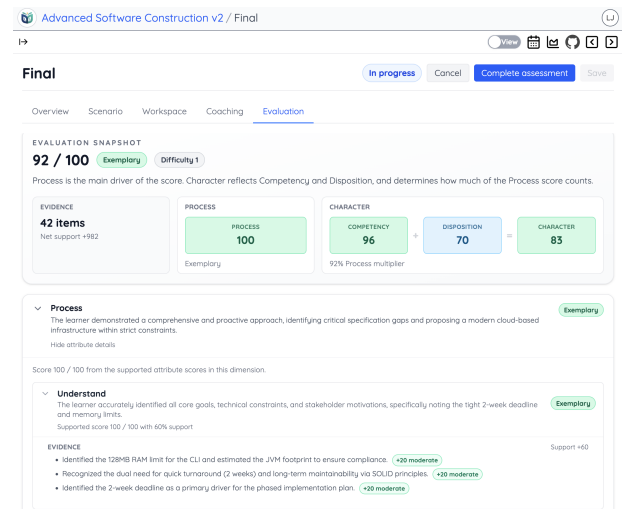


Figure 2b. DRA Evaluation view for the same run, showing the Process, Competency, and Disposition scores and expandable, evidence-backed attribute detail.

[PLACEHOLDER] Still needed: an Interview-specific screenshot (session and evaluation views), a DRA worked example from a non-CS prototype discipline such as Legal, and confirmation of whether CS 260 and CS 329 also used DRA/Interview or only standard interactions.

5.4 Interview: An Applied Specialization

Interview reuses DRA's generative-scenario and evidence-scored evaluation engine for a narrower, more immediately legible use case: job-interview practice. An instructor specifies a discipline, job title, job description, difficulty, learning outcomes, and enabled modes; at runtime the AI generates a multi-session interview with one or more AI-played interviewer personas, and the learner practices across sessions, optionally requesting coaching between them. Evaluation reuses DRA's Competency and Disposition scoring machinery alongside a per-session rating,

combined into a single overall score and rating band using the same evidence-supported calculation. Interview matters to this paper's argument less for its own sake than for what it demonstrates: the DRA evaluation engine is reusable across a structurally different assessment shape, not hand-built for one scenario type, which is independent evidence for the generality of the underlying framework.

[PLACEHOLDER] Figure 3: Interview session and evaluation screenshots. To be supplied.

6 AI-MEDIATED LEARNING INTERACTIONS

Two further features integrate AI into ordinary lesson consumption rather than formal assessment. A topic-aware Discuss panel lets a learner ask questions scoped to the content of the current topic (and, when opened from a specific heading, to that section specifically) without leaving the lesson; useful responses can be saved as personal notes. The teaching interaction type inverts the usual quiz relationship: the learner explains a concept back to the AI, which evaluates the explanation conversationally and scores understanding rather than posing questions to be answered. This design follows evidence that learners who must explain material to a real or simulated learner put in more effort and retain more, an effect known as the protégé effect [6]. Both features are discipline-agnostic: neither assumes code, and both operate purely over the instructional Markdown and learner dialogue.

7 MASTERY TRACKING AND ANALYTICS

Three learner- and instructor-facing views close the loop on the interaction and content model described above. MasteryView summarizes course-level mastery, including completion percentage, time spent, and interaction coverage, for a single learner or, for editors, across an entire class. Progress is a filterable, paginated event log of every recorded action usable for audit or self-review. Metrics aggregates time-on-task and activity trends across courses, learners, and date ranges. An Observe mode lets an editor view a course exactly as a specific learner would, read-only, for support and debugging. These views are also the intended data source for the outcome study we propose as future work (Section 10): the event log already records the granularity a learning-outcomes analysis would require.

8 DEPLOYMENT EXPERIENCE ACROSS DISCIPLINES

We report deployment experience, not a controlled evaluation: we have not yet collected learner-outcome measures or a student- or instructor-satisfaction survey from any MasteryLS course, and we say so explicitly rather than implying rigor we do not have. Deployment experience falls into two tiers that we keep clearly distinct throughout this section.

Real deployment. MasteryLS has been deployed in three live computer science courses at Brigham Young University: CS 260 (Web Programming), CS 240 (Advanced Software Construction), and CS 329 (Quality Assurance and DevOps).

CS 240 has used DRA as a final assessment, illustrated in Figure 2 (Section 5.3).

[PLACEHOLDER] Still needed: one to two sentences confirming which capabilities (standard interactions, AI content generation, github-submission grading, or DRA/Interview) were used in CS 260 and CS 329 specifically.

Prototype exploration. Independent of live deployment, we used MasteryLS's content-generation and DRA/Interview pipelines to build course material and disciplinary scenarios for six additional fields, namely Human-Computer Interaction, Electrical Engineering, Chemical Engineering, Legal, Religious Education, and Agricultural topics, specifically to test whether the tooling holds up outside computer science. None of these six has been deployed to a live course with students; their value is as an existence proof of cross-domain feasibility, not as deployment or outcome evidence. Table 2 summarizes both tiers.

Discipline	Tier	Content	DRA/Int.	Std. Int.
CS 260 (Web Prog.)	Deployed	Yes	?	Yes
CS 240 (Adv. SW Constr.)	Deployed	Yes	Yes	Yes
CS 329 (QA/DevOps)	Deployed	Yes	?	Yes
HCI	Prototype	Yes	?	—
Electrical Eng.	Prototype	Yes	?	—
Chemical Eng.	Prototype	Yes	?	—
Legal	Prototype	?	Yes	—
Religious Ed.	Prototype	Yes	?	—
Agricultural	Prototype	Yes	?	—

Table 2. Discipline $\times$ MasteryLS capability, split by deployment tier. Cells marked “?” are open items to confirm per course/discipline (Section 10, future work); “—” indicates not applicable/not attempted for that combination.

[PLACEHOLDER] Representative content-generation example from a non-CS discipline (e.g., Religious Education or Chemical Engineering) and a DRA/Interview demonstration from one of the deployed CS courses, to be added once available.

We see this two-tier picture as a reasonable, honestly reported first step: three real deployments establish that the system works in production for a real course, and six cross-domain prototypes establish that nothing in the design is CS-specific. Neither tier yet establishes that learners in any discipline learn more, faster, or better because of it, which is the natural next study (Section 10).

9 DISCUSSION

MasteryLS's Git-native content model offers benefits independent of AI: content is portable across platforms, auditable through ordinary Git tooling, and open to multi-author collaboration without a bespoke revision system. Layering AI onto that model, rather than into a separate database-backed authoring surface, keeps AI-authored and AI-graded content subject to the same versioning, diffing, and revert affordances as everything else an instructor writes. The DRA framework is, we think, the paper's strongest evidence that this generality is not merely architectural: real deployment in computer science and structured prototype exploration in six other fields, spanning both technical (Electrical Engineering, Chemical Engineering) and non-technical (Legal, Religious Education) content, is a more convincing existence proof than a design argument alone.

We are equally direct about what that evidence does not show. Deployment in three courses at one institution, and prototype exploration with no live students in six more fields, tells us the system runs and produces plausible content and scenarios; it does

not tell us whether learners master material faster, whether AI-generated feedback is as reliable or fair as instructor feedback, or whether generated DRA scenarios hold up to sustained, discipline-expert scrutiny outside CS. AI grading reliability and bias, academic-integrity questions raised by AI-assisted authoring and AI-assisted submissions, and the security posture of credential handling for GitHub and third-party AI services are all open risks that a systems paper of this scope can describe but not resolve. Instructor-facing controls, namely score override, explicit and inspectable grading criteria, and the autoGrade/syncGrade distinction, are the mechanisms currently in place to mitigate, though not eliminate, these risks, and we view them as a starting point rather than a complete answer.

10 CONCLUSION AND FUTURE WORK

We presented MasteryLS, a Git-native, AI-augmented LMS whose content model, authoring pipeline, and assessment framework are designed to generalize beyond computer science, and reported its deployment in three real BYU CS courses alongside prototype exploration in six additional disciplines. The paper's central artifact is Disciplinary Reasoning Assessment: a generative, evidence-scored simulation framework for authentic professional reasoning assessment, together with its Interview specialization for career-readiness practice. Our priority for future work is closing the evaluation gap this paper is explicit about: a learner-outcome and satisfaction study drawing on the existing CS deployments and, ideally, a first real non-CS deployment converting one of our six prototype disciplines into a live course; and a timed or comparative study of AI-assisted versus manual content authoring. Longer term, we intend to explore adaptive sequencing and richer cross-discipline analytics built on the progress data the system already records.

ACKNOWLEDGMENTS AND AI-USE DISCLOSURE

Generative AI (Anthropic Claude) was used to assist with drafting and organizing this manuscript from the author's design documentation and prior notes. All technical claims were reviewed by the author, who takes full responsibility for the content, consistent with ACM's policy on authorship and generative-AI use.

REFERENCES

- [1] Benjamin S. Bloom. 1968. Learning for Mastery. *Evaluation Comment* 1, 2 (1968), 1–12.
- [2] Howard S. Barrows and Stephen Abrahamson. 1964. The Programmed Patient: A Technique for Appraising Student Performance in Clinical Neurology. *Journal of Medical Education* 39, 8 (1964), 802–805.
- [3] Nishat Raihan, Mohammed Latif Siddiq, Joanna C. S. Santos, and Marcos Zampieri. 2025. Large Language Models in Computer Science Education: A Systematic Literature Review. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V.1* (SIGCSE TS 2025). ACM.
- [4] Lassi Haaranen and Teemu Lehtinen. 2015. Teaching Git on the Side: Version Control System as a Course Platform. In *Proceedings of the 2015 ACM Conference on Innovation and Technology in Computer Science Education* (ITiCSE '15). ACM, 87–92.
- [5] Joseph Feliciano, Margaret-Anne Storey, and Alexey Zagalsky. 2016. Student Experiences Using GitHub in Software Engineering Courses: A Case Study. In *Proceedings of the 38th International Conference on Software Engineering Companion* (ICSE-C '16). ACM, 422–431.

- [6] Catherine C. Chase, Doris B. Chin, Marily A. Oppezzo, and Daniel L. Schwartz. 2009. Teachable Agents and the Protégé Effect: Increasing the Effort Towards Learning. *Journal of Science Education and Technology* 18 (2009), 334–352.